LAB 4: Shell Scripting Evaluation 1A

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer,CSE
[2]Junior Lecturer,CSE

December 8, 2025

General Instructions

1. Create a separate file for each task (e.g., ID_task1.sh, ID_task2.sh etc.).
2. Use the PC from the lab.
3. Submit your mobile at the front.

Task 1: Find the Greatest Element in an Array (Loops and Arrays)

Concepts Practiced: Arrays, loops, conditional statements, arithmetic comparisons, user input.
Goal: Write a shell script that reads array elements from the user, and then finds and displays the greatest element in the array.

Requirements

1. Prompt the user to enter the number of elements in an array.
2. Read all array elements from the user.
3. Find and display the greatest element in the array.

Hints

- Use a loop (`for` or `while`) to read array elements from the user.
- Initialize a variable (e.g., `max`) with the first element of the array.
- Use a loop to compare each element with `max` and update `max` if a larger element is found.
- Use `echo` to display the greatest element.
- Consider using double quotes around array values (e.g., `"${array[i]}"`) to safely handle spaces or special characters.

Expected Output (Example)

```
Enter the number of elements: 5
Enter element 1: 15
Enter element 2: 7
Enter element 3: 42
Enter element 4: 10
Enter element 5: 28
The greatest element in the array is: 42
```

Task 2:

Objective

In this labtask, you will write a Bash script to automate the build process of a C++ project titled "Vid2Console".

While the project is written in C++, **you do not need to know how to write C++ code**. Your task is to create a script that installs necessary dependencies, manages build directories, handles compilation flags (Debug/Release), and links the final executable. To start with, clone the following **Git Repo :**

```
    git clone -b lab4 https://github.com/azfarus/vid2console.git
```

1. Project Structure

Assume you are in the root directory of the project. The file structure is as follows:

```
project_root/
  include/                # Header files (.h)
  lib/                    # Library implementation files (.cpp)
     printer.cpp
     renderer.cpp
     vid2arr.cpp
     framegenerator.cpp
  src/
     vid2console/
        main.cpp          # The entry point
  build.sh                # YOU are creating this file
```

2. Compilation Basics (Read Carefully)

Since we are manually invoking the compiler (g++), you need to understand the pipeline. C++ compilation happens in two distinct stages:

Stage A: Compiling to Objects (.o)

We take a source file (`.cpp`) and turn it into a binary object file (`.o`).

- **The Command:** `g++ -c source.cpp -o output.o`
- **Flag -c:** Tells g++ to "compile only, do not link yet."

- **Flag -I <path>:** The compiler needs to know where the header files (`.h`) live. Since our headers are in the `include` folder, we must pass `-I include`.
- **External Libs (OpenCV):** This project uses OpenCV. Instead of guessing where OpenCV is installed, we use a tool called `pkg-config`.
  - To get compilation flags: `pkg-config --cflags opencv4`

Stage B: Linking (.exe or binary)

We take *all* the `.o` files created in Stage A and glue them together to create the final executable.

- **The Command:** `g++ file1.o file2.o ... -o final_executable`
- **Linking Libraries:** We need to tell the linker to include the actual OpenCV library code.
  - To get linker flags: `pkg-config --libs opencv4`

3. Task Requirements & Steps

Create a file named `build.sh` and implement the following:

Step 1: Safety and Dependencies

You must start the script by ensuring it exits on errors and installing the necessary tools.

Required Implementation for Step 1: Copy the following code exactly to start your script. These lines will install the required dependancies for compilation. :

```
#!/usr/bin/bash
set -e #sets the script to exit when any error occurs

# Update package lists
sudo apt update

# Install ffmpeg and related libraries
sudo apt install -y ffmpeg libavcodec-dev libavformat-dev libavutil-dev
    libswscale-dev

# Install OpenCV and development headers
sudo apt install -y libopencv-dev

# Install Build Essentials (GCC, G++, Make, etc.)
sudo apt install -y build-essential
```

Step 2: Variable Definitions

Define variables at the top of your script (after the installations) to keep it clean.

1. **Directories:** Define variables for an Object directory (`build/obj`) and a Binary directory (`build/bin`).
2. **OpenCV Flags:**
   - Create a variable `OPENCV_COMPILE_FLAGS` that stores the result of `$(pkg-config --libs opencv4 --cflags opencv4)`.

- Create a variable `OPENCV_LINKING_FLAGS` that stores `$(pkg-config --libs opencv4)`.

3. **Standard Flags:** Initialize a variable `CXX_FLAGS` as an empty string.

Step 3: Argument Parsing (Loops and Conditionals)

Your script should accept command-line arguments (e.g., `./build.sh clean debug`). Iterate through all arguments passed to the script (run a for loop through `"$@"`) and implement the following logic:

1. **Clean:** If the argument is `clean`:
   - Check if the `build` directory exists.Use the syntax `if !(ls build)`
   - If it does, remove it.
2. **Debug:** If the argument is `debug`:
   - Set `CXX_FLAGS` to `-g -O0` (Includes symbols, no optimization).
3. **Release:** If the argument is `release`:
   - Set `CXX_FLAGS` to `-s -O3` (Strip symbols, maximum optimization).

Step 4: Directory Preparation

After checking arguments, ensure your build directories (`build/obj` and `build/bin`) exist.

Hint: Use `mkdir` with the flag that creates parents if needed and doesn't complain if they already exist.

Step 5: The Compilation Phase

You need to compile 5 specific files. Print a message to the console saying "Compiling...".

General Syntax for this step:

```

g++ -c $CXX_FLAGS <source_file> -o <output_object> -I <include_dir> \
    $OPENCV_COMPILE_FLAGS -std=c++17
```

Your Task: Write the specific commands to compile the following mappings:

Source File (.cpp)	Output Object (.o) inside build/obj
src/vid2console/main.cpp	main.o
lib/printer.cpp	printer.o
lib/renderer.cpp	renderer.o
lib/vid2arr.cpp	vid2arr.o
lib/framegenerator.cpp	framegenerator.o

Step 6: The Linking Phase

Now, link all those object files into one executable. Print a message saying "Linking...".

Syntax:

```

g++ <list of all .o files> $OPENCV_LINKING_FLAGS -o <final_binary_path> -
    std=c++17
```

1. The final binary should be placed in `build/bin/vid2console`.
2. After linking, use `chmod` to ensure the resulting binary has execute permissions (777).

4. Testing Your Script

Run the following commands to verify your script works:

1. **Clean build:** `./build.sh clean` (Should remove build folder)
2. **Debug build:** `./build.sh debug` (Should build without errors)
3. **Run it:** `./build/bin/vid2console`